

# So Performative

---

Lab 7 · Performance

# Buffer Slide

---

- Doubts from last week's lab? Ask now.
- **No labexam this week.**
- Outlab: status and doubts (TA update).
- This lab is **ungraded** (0 marks).

# Plan for this lab

---

- Part 1: Understanding MIPS64
- Part 2: Reviewing pipelining and unrolling
- Part 3: Learning perf
- Part 4: Timing code with chrono, rdtsc and perf

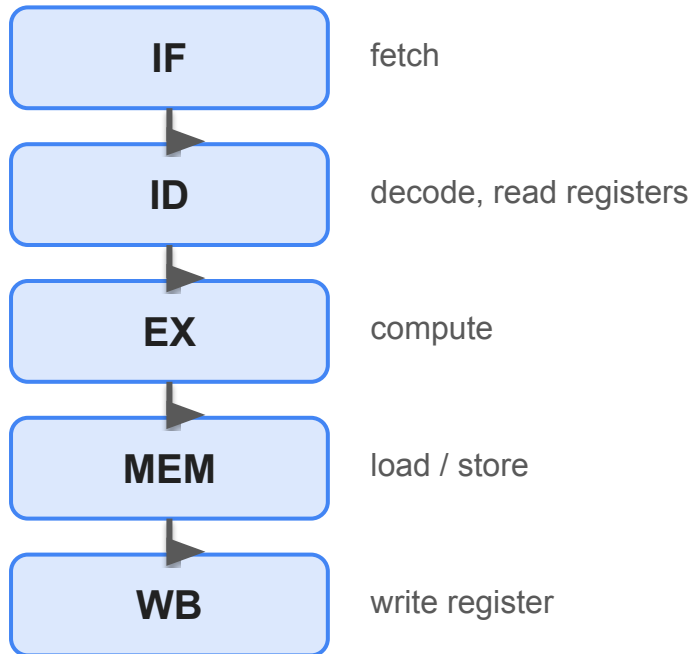
## Part 1

# Understanding MIPS64

- 
- EduMIPS64
  - MIPS64 syntax
  - Using the simulator

# Meet EduMIPS64

- A simulator of the classic 5-stage MIPS64 pipeline.
- Shows every instruction moving through the stages, cycle by cycle.
- Also shows registers, memory and statistics (cycles, CPI, stalls).
- Forwarding is a switch: turn it off and on, watch stalls come and go.
- No branch predictor, no out-of-order execution, at most one instruction per cycle.



Read more: <https://edumips64.readthedocs.io/en/latest/> · <https://web.edumips.org>

# Two ways to run it

|               | Web UI                                                                                 | Jar + makefile                                                              |
|---------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Install       | nothing: open <code>web.edumips.org</code>                                             | Needs Java 17+, run as <code>java -jar &lt;jarfile&gt;</code>               |
| Load code     | type or paste, or <code>Program</code> → <code>Open...</code> , then <code>Load</code> | <code>make task1</code> loads the file for you                              |
| Run           | toolbar buttons                                                                        | <code>make task1</code> , <code>make task2</code> , <code>make task3</code> |
| Big tests     | paste a test from <code>tests.txt</code>                                               | generated and run automatically                                             |
| Check answers | read <code>c</code> in the Memory panel                                                | <code>make check</code> prints PASS / FAIL                                  |
| Best for      | stepping and looking                                                                   | counting and checking                                                       |

**Both run the same simulator. Second one is easy to use on CLI, first one will always work since its Web UI**

Read more: <https://web.edumips.org> · <https://github.com/EduMIPS64/edumips64/releases>

# Getting the jar running

---

- Install Java: a JRE 17, 21 or newer. How depends on your system, ask GPT... Ask a TA if you get stuck.
- Check it: `java -version` must say 17 or higher.
- The jar is already in `07_inlab/`. You can get a copy from:
  - [github.com/EduMIPS64/edumips64/releases/download/v1.4.0/edumips64-1.4.0.jar](https://github.com/EduMIPS64/edumips64/releases/download/v1.4.0/edumips64-1.4.0.jar)
- To open desktop window: `java -jar edumips64-1.4.0.jar`
- Headless, from the makefile: `make task1`
- The jar calls itself 1.4.0-alpha. That is expected.

# EduMIPS64 troubleshooting

| Symptom                                                                   | Fix                                                                                 |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>java: command not found</code>                                      | Install Java 17 or newer                                                            |
| <code>UnsupportedClassVersionError</code> (class file version 61)         | Java is older than 17: install a newer one                                          |
| Jar does not start                                                        | Run it from a terminal to see the error: <code>java -jar edumips64-1.4.0.jar</code> |
| <code>Unable to parse file / Unknown directive in the data section</code> | An instruction sits before <code>.text</code> . Move it below.                      |
| Warning <code>No terminating instruction detected</code>                  | Add <code>syscall 0</code> at the end                                               |
| Forwarding setting will not change                                        | Stop the program first. It applies after a reset.                                   |
| Web UI page stays blank                                                   | Try another browser                                                                 |
| Still stuck                                                               | Ask a TA                                                                            |

# MIPS64 syntax: the building blocks

| Data                         | Meaning                    |
|------------------------------|----------------------------|
| <code>.data / .text</code>   | start data / code section  |
| <code>.word 5, 0x10</code>   | 64-bit values              |
| <code>.word32 .word16</code> | 32-bit / 16-bit values     |
| <code>.byte 1, 2</code>      | bytes                      |
| <code>.space 64</code>       | 64 zeroed bytes            |
| <code>.asciiz "hi"</code>    | string, ends with a 0 byte |
| <code>.ascii "hi"</code>     | string, no 0 byte          |

| Code                    | Meaning                                                    |
|-------------------------|------------------------------------------------------------|
| <code>name:</code>      | label                                                      |
| <code>; text</code>     | comment                                                    |
| <code>r0 ... r31</code> | registers; <code>\$4</code> and <code>\$t0</code> work too |
| <code>r0</code>         | always reads 0                                             |
| <code>5, 0x1F</code>    | immediate: decimal or hex                                  |
| <code>8(r2)</code>      | memory at <code>r2 + 8</code>                              |
| <code>a(r4)</code>      | memory at label <code>a</code> + <code>r4</code>           |
| <code>syscall 0</code>  | stop the program                                           |

> Read the manual attached in the **07\_inlab** folder for detailed information

# Common MIPS64 idioms

Prefix **d** means this works on 64-bit regs

| Code                                                | What it does                                   |
|-----------------------------------------------------|------------------------------------------------|
| <code>daddi r1, r0, 5</code>                        | load a constant: $r1 = 5$                      |
| <code>dadd r2, r1, r0</code>                        | copy: $r2 = r1$                                |
| <code>dadd r3, r0, r0</code>                        | zero: $r3 = 0$                                 |
| <code>daddi r1, r0, a</code>                        | $r1$ = address of label <code>a</code>         |
| <code>ld r5, 0(r1) then daddi r1, r1, 8</code>      | read one element, step to the next 64-bit word |
| <code>daddi r4, r4, -1 then bne r4, r0, loop</code> | count down, loop until $r4$ is 0               |
| <code>slti r8, r4, 4 then bne r8, r0, few</code>    | jump to <code>few</code> if $r4 < 4$           |
| <code>dsl1 r5, r4, 3</code>                         | $r5 = r4 * 8$ (shift left by 3)                |
| <code>syscall 0</code>                              | stop                                           |

Use `bne` / `beq` with `r0...` `bnez` and `beqz` work but with a warning so avoid them.

# The web UI tour

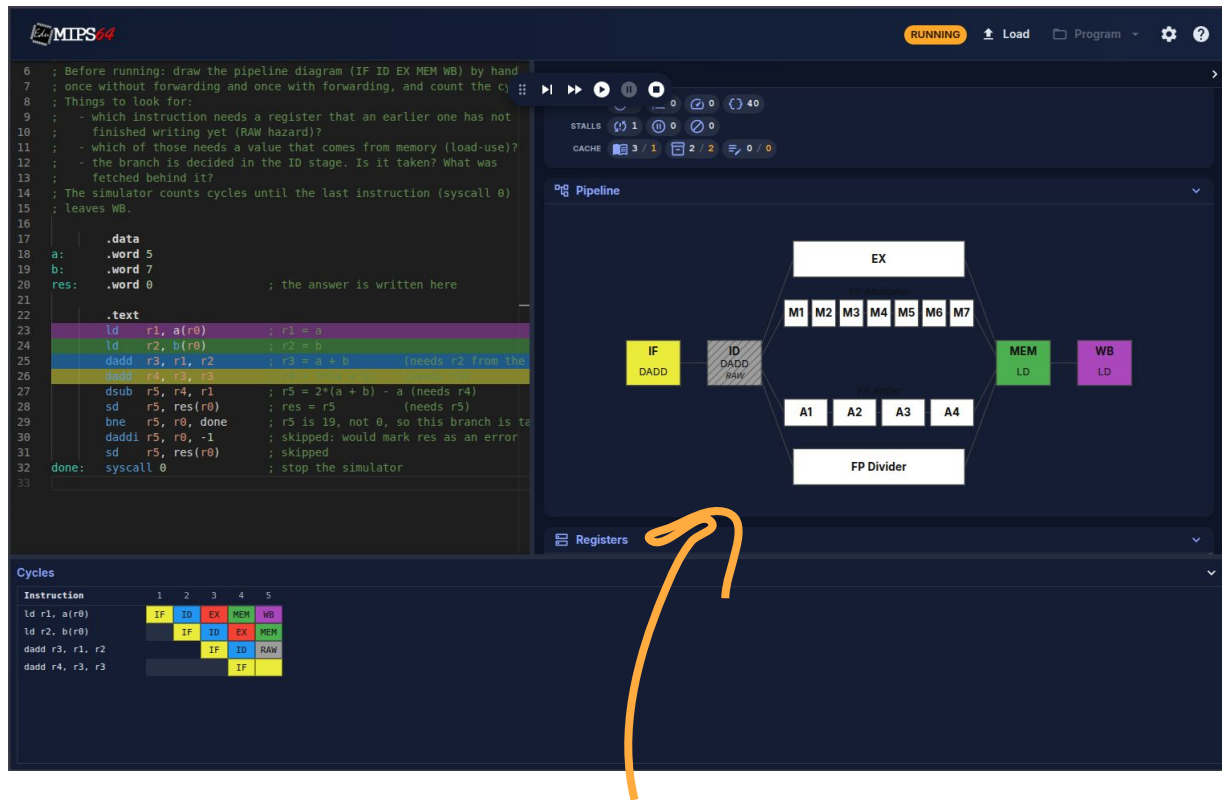
- Bottom: the cycles diagram. One row per instruction, one column per cycle.

- Panels: Pipeline, Registers, Memory, Statistics, Standard Output.

- Toolbar: single step, multi step, run all, stop. Reset starts over.

- Program → Open... loads a file. Load assembles it.

- Gear icon → General Settings → CPU Forwarding.



Scroll down here to see Registers, Memory, Standard Output

# The jar (desktop) UI tour

- **Windows:** Cycles, Pipeline, Registers, Data, Statistics.
- File → Open loads a program.
- Execute: Single Cycle (F7), Multi-Cycle (F8), Run (F4), Stop (F9).
- Configure → Settings → Main settings → Enable forwarding.

The screenshot displays the EduMIPS64 v. 1.4.0-alpha-1119711 MIPS64 Processor Simulator interface, which is divided into several functional panels:

- File Bar:** Located at the top left, it includes standard menu options: File, Execute, Configure, Tools, Window, and Help.
- Cycles Panel:** On the upper left, it shows assembly instructions being executed:
 

```
ld r1,a(r0)
ld r2,b(r0)
dadd r3,r1,r2
dadd r4,r3,r3
```

 A pipeline diagram below the code visualizes the stages (IF, ID, EX, MEM, WB) for each instruction across multiple cycles.
- Registers Panel:** In the upper middle, it lists the state of all 13 registers (R0-R13), showing their names (e.g., \$zero, \$at), hexadecimal values, and flags like C, Z, O, N, D.
- Statistics Panel:** On the upper right, it provides performance metrics such as "Execution" (6 Cycles, 1 Instruction, 6.0 CPI) and "Stalls" (2 RAW Stalls, 0 WAW Stalls). It also details structural stalls and cache memory statistics.
- Pipeline Panel:** On the lower left, it features a detailed block diagram of the processor's internal components, including the FP Multiplier, FP Adder, FP-DIV 0 unit, and various buffers (DADD, LD).
- Data Panel:** In the lower middle, it presents a table mapping memory addresses to their representations and labels, such as:
 

| Address | Representation   | Label     |
|---------|------------------|-----------|
| 0000    | 0000000000000005 | a .word   |
| 0008    | 0000000000000007 | b .word   |
| 0010    | 0000000000000000 | res .word |
- Code Panel:** On the lower right, it shows the disassembled machine code corresponding to the assembly instructions, listing address, representation, and label:
 

| Address | Representation | Label      |
|---------|----------------|------------|
| 0000    | DC010000       | ld r1      |
| 0004    | DC020008       | ld r2      |
| 0008    | 0022182C       | dadd       |
| 000C    | 0063202C       | dadd       |
| 0010    | 0081282E       | sdsb       |
| 0014    | FC050010       | sd r5      |
| 0018    | 14A00002       | bne r      |
| 001C    | 6005FFFF       | daddi      |
| 0020    | FC050010       | sd r5      |
| 0024    | 0000000C       | DONE sysca |
| 0028    |                |            |
| 002C    |                |            |
| 0030    |                |            |

At the bottom of the window, a status bar indicates the current state: "Welcome to EduMIPS64 1.4.0-alpha-1119711" on the left and "RUNNING" on the right.

# Forwarding switch and Statistics

| Where    | Forwarding switch                                           |
|----------|-------------------------------------------------------------|
| Web UI   | gear icon → General Settings → CPU Forwarding               |
| Desktop  | Configure → Settings → Main settings → Enable forwarding    |
| makefile | <code>make task1 args=fwd</code> or <code>args=nofwd</code> |

- Statistics: cycles, instructions, CPI = cycles / instructions, RAW stalls.
- A taken branch shows as a bubble in the cycles diagram and in the total. The desktop Statistics window has no branch counter.
- CPI includes the 4-cycle fill. Compare total cycles.

## Part 2

# Reviewing pipelining and unrolling

- 
- The 5-stage pipeline
  - Hazards and forwarding
  - Loops and unrolling
  - Code for any N

# Five stages

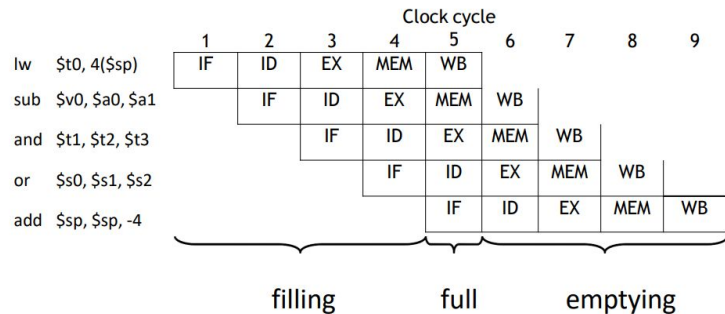
| Stage | What happens                            |
|-------|-----------------------------------------|
| IF    | fetch the instruction                   |
| ID    | decode, read registers, decide branches |
| EX    | the ALU computes                        |
| MEM   | load or store                           |
| WB    | write the result to the register        |

One instruction per stage.

Every cycle, each one moves one stage down.

Read more: [https://en.wikipedia.org/wiki/Classic\\_RISC\\_pipeline](https://en.wikipedia.org/wiki/Classic_RISC_pipeline)  
<https://www.cse.iitb.ac.in/~biswa/courses/CS230/schedule.html>

## Visualizing Pipeline: Execution time



For a k-stage pipeline executing N instructions

first instruction: K cycles

Next N-1 instructions: N-1 cycles, total =  $K + (N-1)$  cycles

# In a perfect world

---

- One instruction alone needs 5 cycles.
- With no waiting,  $n$  instructions need  $4 + n$  cycles.
- $\text{CPI} = \text{cycles} / \text{instructions}$ .
- The first 4 cycles fill the pipeline, so CPI reaches 1 only for large  $n$ .

| n   | cycles | CPI  |
|-----|--------|------|
| 1   | 5      | 5.00 |
| 4   | 8      | 2.00 |
| 100 | 104    | 1.04 |

Read more: [https://en.wikipedia.org/wiki/Cycles\\_per\\_instruction](https://en.wikipedia.org/wiki/Cycles_per_instruction)

# Read after write: you know this stuff, don't you?

- RAW hazard: an instruction reads a register an earlier one writes.
- The producer writes in WB, the consumer reads in ID.
- No forwarding: a read right behind waits 2, a read with one instruction in between waits 1, a read with two or more instructions in between do not wait

| cycle                      | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|----------------------------|----|----|----|----|----|----|----|----|----|----|
| <code>daddi r1,r0,5</code> | IF | ID | EX | ME | WB |    |    |    |    |    |
| <code>dadd r2,r1,r1</code> |    | IF | ID | s  | s  | EX | ME | WB |    |    |
| <code>daddi r3,r0,7</code> |    |    | IF | .  | .  | ID | EX | ME | WB |    |
| <code>syscall 0</code>     |    |    |    |    |    | IF | ID | EX | ME | WB |

s = waiting in ID

. = held in IF

**No forwarding: 10 cycles**

**Forwarding: 8 cycles**

Read more: [https://en.wikipedia.org/wiki/Hazard\\_\(computer\\_architecture\)](https://en.wikipedia.org/wiki/Hazard_(computer_architecture))

# Forwarding: skip the register file

- Forwarding (bypassing): a later instruction takes a value from a pipeline register, before WB writes it.
- ALU result → next instruction: 0 cycles.
- `ld` then use the loaded value: 1 cycle. The value exists only after MEM.
- `sd` storing a just-loaded value: 0 cycles.

## Data Hazard Detector and stalls

Execute to decode:

$EX/MEM.RegisterRd = ID/EX.RegisterRs$

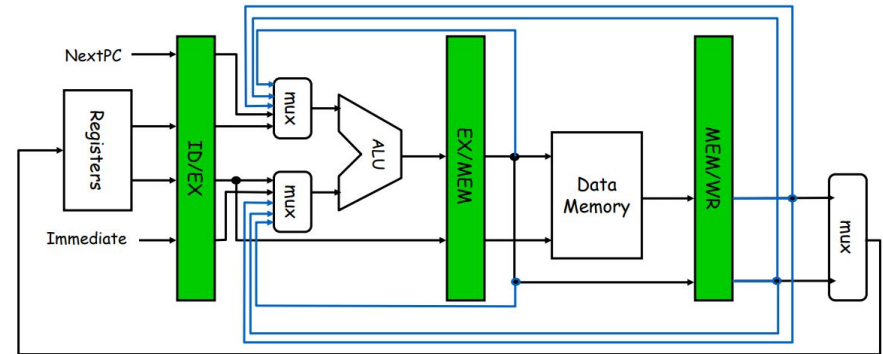
$EX/MEM.RegisterRd = ID/EX.RegisterRt$

Memory to decode:

$MEM/WB.RegisterRd = ID/EX.RegisterRs$

$MEM/WB.RegisterRd = ID/EX.RegisterRt$

## Bypassing/forwarding: Updated Datapath



Read more: [https://en.wikipedia.org/wiki/Operand\\_forwarding](https://en.wikipedia.org/wiki/Operand_forwarding)  
<https://www.cse.iitb.ac.in/~biswa/courses/CS230/schedule.html>

# Branches: take it or leave it

- A branch is decided in ID.
- Meanwhile IF fetches the next instruction in order.
- Taken: that instruction is thrown away.  
1 cycle lost.
- Not taken: 0.
- A branch reads its register in ID, so it waits like any consumer.
  - Question for you: What is a branch delay slot and how can it help here?

| Branch right after ...       | no fwd | fwd |
|------------------------------|--------|-----|
| ALU instruction              | 2      | 1   |
| ld                           | 2      | 2   |
| ld, one instruction between  | 1      | 1   |
| ALU, one instruction between | 1      | 0   |
| taken branch (bubble)        | 1      | 1   |

Stall cycles. Full rules: Table 2 at the end of the inlab PDF.

# Mommy, I can count!

**$\text{cycles} = 4 + \text{instructions} + \text{stall cycles} + \text{branch bubbles}$**

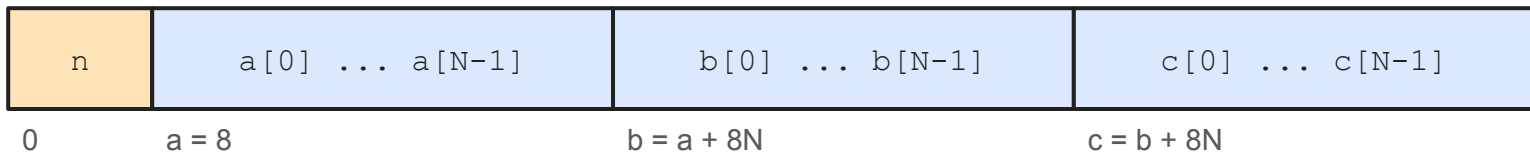
- `syscall 0` counts as an instruction.
- Counting ends when `syscall 0` leaves WB.
- Task 1: draw `hazard.s` by hand, both ways, then check.

| hazard.s         | no fwd       | fwd   |
|------------------|--------------|-------|
| fill             | 4            | 4     |
| instructions     | 8            | 8     |
| stalls + bubbles | ? (you find) | ?     |
| cycles           | 21           | 14    |
| CPI              | 2.625        | 1.750 |

Read more: [https://en.wikipedia.org/wiki/Classic\\_RISC\\_pipeline](https://en.wikipedia.org/wiki/Classic_RISC_pipeline)

**It's Task 1 time: count on it**

# Test data and the web simulator



For task2/3 we have tests, if you're running the jar just do `make task2/3`. Otherwise you need to run:

- `make task2-tests` / `make task3-tests` : they write `task2/tests.txt` / `task3/tests.txt`.
- Open the file, go to the web simulator, paste one test over the data section: from `.data` up to, not including, `.text`.
- Run, then compare `c` in the Memory panel. Values there are hex.
- `make check` does all of this for you with the jar.

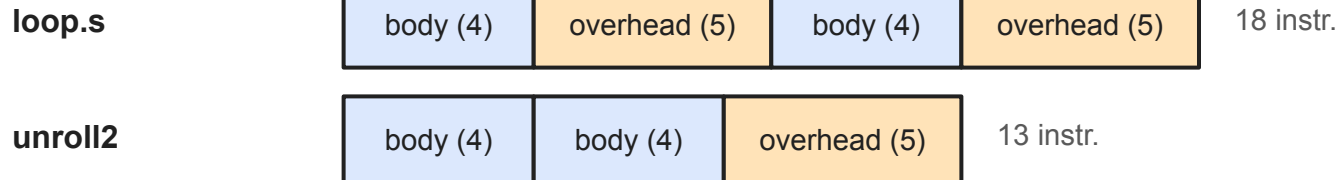
# Going around in circles

| loop.s body                   | role       |
|-------------------------------|------------|
| <code>ld r5, 0(r1)</code>     | load a[i]  |
| <code>ld r6, 0(r2)</code>     | load b[i]  |
| <code>dadd r7, r5, r6</code>  | add        |
| <code>sd r7, 0(r3)</code>     | store c[i] |
| <code>daddi r1, r1, 8</code>  | overhead   |
| <code>daddi r2, r2, 8</code>  | overhead   |
| <code>daddi r3, r3, 8</code>  | overhead   |
| <code>daddi r4, r4, -1</code> | overhead   |
| <code>bne r4, r0, loop</code> | overhead   |

- 9 instructions per element: 4 do the work, 5 run the loop.
- Cost per iteration: 16 cycles without forwarding, 12 with.
- `make task2 on loop.s`:
  - N = 8: 142 / 105 cycles
  - N = 4096: 65550 / 49161 cycles

# We (un)rollin' – what is it even about

- Unroll by k: write the body k times in one iteration.
- Pointers and counter **move once** per iteration, by k elements.
- Offsets  $0(r1)$ ,  $8(r1)$ ,  $16(r1)$  ... and fresh registers per copy.
- This **amortizes** (did you learn this word in DSA?) the loop overhead but does not fix any data hazards
- Think why...



Two elements (a, b). Cycles per element at N = 4096: **loop.s 16.0 / 12.0** (no fwd / fwd)  
**unroll2 12.0 / 8.5** (no fwd / fwd).

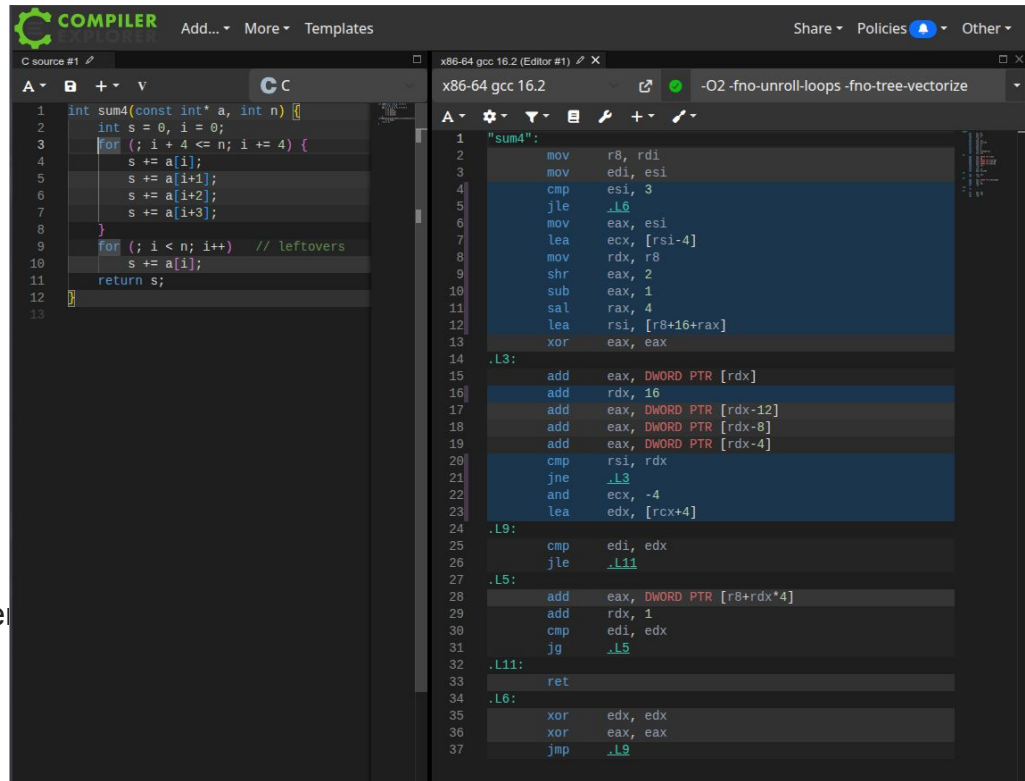
Read more: [https://en.wikipedia.org/wiki/Loop\\_unrolling](https://en.wikipedia.org/wiki/Loop_unrolling)

# Manual unrolling code looks like this in C

```
int sum4(const int* a, int n) {
    int s = 0, i = 0;
    for (; i + 4 <= n; i += 4) {
        s += a[i];
        s += a[i+1];
        s += a[i+2];
        s += a[i+3];
    }
    for (; i < n; i++)    // leftovers
        s += a[i];
    return s;
}
```

- Main loop: it does four **adds** + one pointer step together
- After it: a one-**add** loop for the last  $n \% 4$  elements
- Flags: **-O2 -fno-unroll-loops -fno-tree-vectorize**

[Link](#)



The screenshot displays a compiler interface with two panels. The left panel shows the C source code for a function `sum4` that calculates the sum of an array. The right panel shows the corresponding assembly code generated by the compiler. The assembly code is for x86-64 gcc 16.2 and includes flags `-O2 -fno-unroll-loops -fno-tree-vectorize`. The assembly code shows the main loop unrolled into a series of `add` instructions for the first four elements, followed by a loop for the remaining elements. The assembly code is as follows:

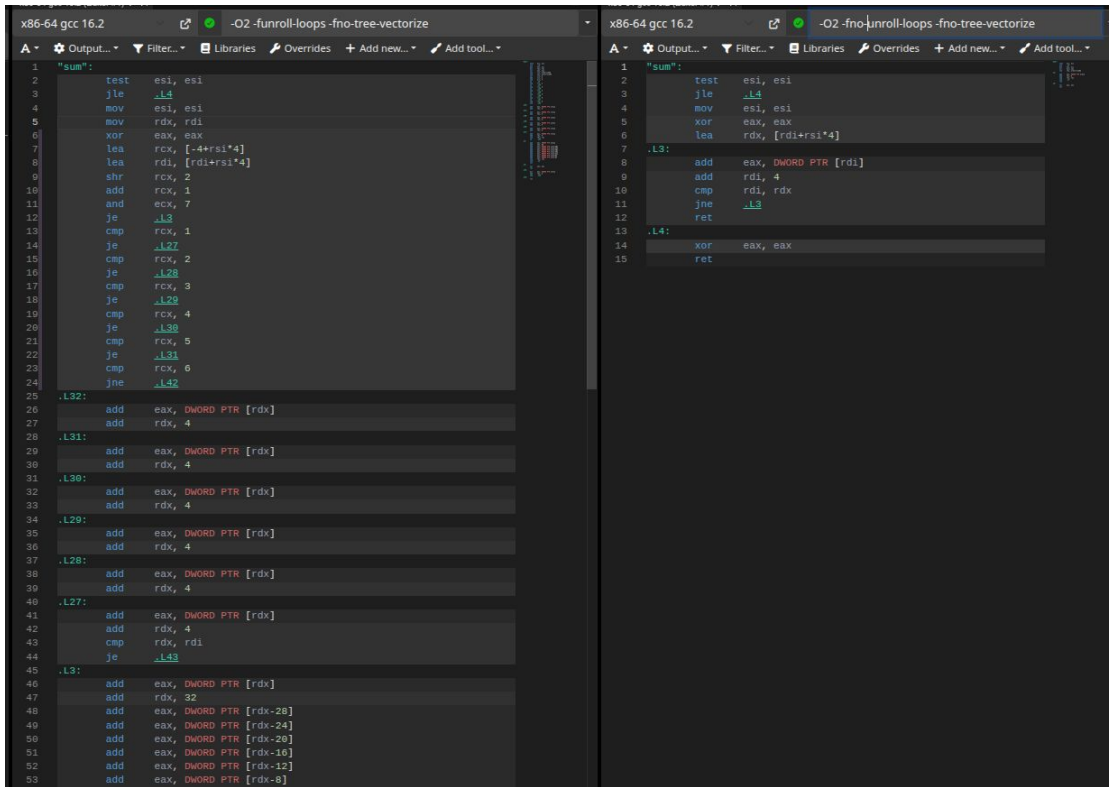
```
1  "sum4":
2      mov     r8, rdi
3      mov     edi, esi
4      cmp     esi, 3
5      jle     .L6
6      mov     eax, esi
7      lea     ecx, [rsi-4]
8      mov     rdx, r8
9      shr     eax, 2
10     sub     eax, 1
11     sal     rax, 4
12     lea     rsi, [r8+16+rax]
13     xor     eax, eax
14
15     .L3:
16     add     eax, DWORD PTR [rdx]
17     add     rdx, 16
18     add     eax, DWORD PTR [rdx-12]
19     add     eax, DWORD PTR [rdx-8]
20     add     eax, DWORD PTR [rdx-4]
21     cmp     rsi, rdx
22     jne     .L3
23     and     ecx, -4
24     lea     edx, [rcx+4]
25
26     .L9:
27     cmp     edi, edx
28     jle     .L11
29
30     .L5:
31     add     eax, DWORD PTR [r8+rdx*4]
32     add     rdx, 1
33     cmp     edi, edx
34     jg      .L5
35
36     .L11:
37     ret
38
39     .L6:
40     xor     edx, edx
41     xor     eax, eax
42     jmp     .L9
```

# Compilers are smarter than you

```
int sum(const int* a, int n) {  
    int s = 0;  
    for (int i = 0; i < n; i++)  
        s += a[i];  
    return s;  
}
```

- Flags: `-O2 -funroll-loops -fno-tree-vectorize`
- Compare: the same, with `-fno-unroll-loops`.
- (LEFT) The compiler unrolls MULTIPLE LEVELS for you, so you rarely write this by hand.
- It unrolls by 4, or 2, or 1 (leftovers) etc.
- (RIGHT) basic loop

[Link](#)



# How to increase CPI (but we don't want that here)

| N = 8, no forwarding               | instructions | cycles | CPI   |
|------------------------------------|--------------|--------|-------|
| <code>loop.s</code>                | 78           | 142    | 1.821 |
| <code>unroll2.s</code> (reference) | 58           | 110    | 1.897 |
| <code>unroll4.s</code> (reference) | 48           | 94     | 1.958 |

- Unrolling removes **cheap instructions** (pointer and counter updates, branches).
- The stalls stay, so CPI goes up while total cycles go down.
- Judge speed by total cycles for the same work.

Isn't more CPI bad? hmm

# When to unroll, when to hold

## Helps

- Loop overhead: counter, pointers
- Fewer branches
- Many iterations
- Introduces parallel fetches (we will learn this later)

## Does little for

- A dependency chain through one accumulator (`s += a[i]`)
- Loops that are memory-bandwidth bound (we will learn this later)

## Costs

- More code!
- More instructions, larger program, more stress on i-cache
- Can not always be done easily; here it only works directly because all iterations do independent work.

# Bigger code, fewer laps

| unroll                    | instr / iteration | overhead / element | cycles / element N = 8<br>(nofwd / fwd) | cycles / element N = 4096<br>(nofwd / fwd) |
|---------------------------|-------------------|--------------------|-----------------------------------------|--------------------------------------------|
| 1 ( <code>loop.s</code> ) | 9                 | 5                  | 17.8 / 13.1                             | 16.0 / 12.0                                |
| 2                         | 13                | 2.5                | 13.8 / 9.6                              | 12.0 / 8.5                                 |
| 4                         | 21                | 1.25               | 11.8 / 7.9                              | 10.0 / 6.75                                |

- Overhead is 5 instructions per iteration in every version.
- Cycles are no fwd / fwd, from `make task2` with the reference solutions.
- Short arrays are more affected by setup/fill costs; long arrays expose the steady-state loop performance.

**It's Task 2 time: we rollin**

# Finish your leftovers

- An unrolled loop handles  $k$  elements per pass.
- If  $N$  is not a multiple of  $k$ , some elements are left over.

**N = 9**



**N = 6**

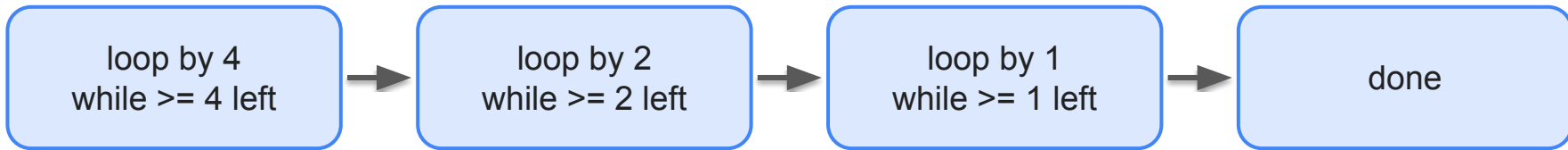


blue: loop by 4   green: loop by 2   yellow: loop by 1  
 $2047 = 511 \times 4 + 1 \times 2 + 1$

| N    | by 4 | by 2 | by 1 |
|------|------|------|------|
| 9    | 2    | 0    | 1    |
| 6    | 1    | 1    | 0    |
| 2047 | 511  | 1    | 1    |

# Clean your plate

---



- After the loop by 4, 0 to 3 elements remain.
- So the loop by 2 runs at most once, and the loop by 1 at most once.
- Compilers emit all these shapes and goes through them one by one.
- Each loop needs two decisions: enter at all? go around again?

# Should I stay or should I go?

| Code                              | Meaning                             |
|-----------------------------------|-------------------------------------|
| <code>slti rd, rs, imm</code>     | rd = 1 if rs < imm (signed), else 0 |
| <code>slt rd, rs, rt</code>       | rd = 1 if rs < rt (signed), else 0  |
| <code>bne rd, r0, L</code>        | jump if the condition held          |
| <code>beq rd, r0, L</code>        | jump if it did not                  |
| <code>dsub then bgez rs, L</code> | jump if a difference is $\geq 0$    |

- EduMIPS64 has no `bltz`, `blez` or `bgtz`.
- Idea: do the comparison with `slti/slt` then branch on the result

# Tips for hand-optimizing assembly (in terms of hazards)

---

- TIP: Keep at least **two independent instructions** between a producer and its consumer. This avoids stalls, with or without forwarding.
- A store reads **both** its data and address registers: `sd r7, 0(r3)` reads `r7` and `r3`.
- Branches read their operands in **ID** but decide branch in **EX**, so compute the comparison early enough to avoid a branch-data hazard.
- **Taken branches** incur a cycle of penalty. Fewer taken branches means fewer wasted cycles.
- **Correctness comes first**. A faster program that fails even one test case is worthless.

# Any N, any time

| N    | leftover | tests                 |
|------|----------|-----------------------|
| 8    | 0        | short, multiple of 4  |
| 6    | 2        | short, loop by 2      |
| 9    | 1        | short, loop by 1      |
| 1024 | 0        | long, multiple of 4   |
| 2047 | 3        | long, both remainders |
| 4098 | 2        | long, loop by 2       |

| N    | loop.s        | reference general.s |
|------|---------------|---------------------|
| 8    | 142 / 105     | 72 / 64             |
| 6    | 110 / 81      | 62 / 54             |
| 9    | 158 / 117     | 80 / 72             |
| 1024 | 16398 / 12297 | 5660 / 5652         |
| 2047 | 32766 / 24573 | 11290 / 11282       |
| 4098 | 65582 / 49185 | 22568 / 22560       |

Short arrays show the setup and leftover cost; long arrays show the main steady-state loop.  
Right table: cycles no fwd / fwd from `make task3`. Reference results to compare with your own.

**It's Task 3 time: leave no element  
behind**

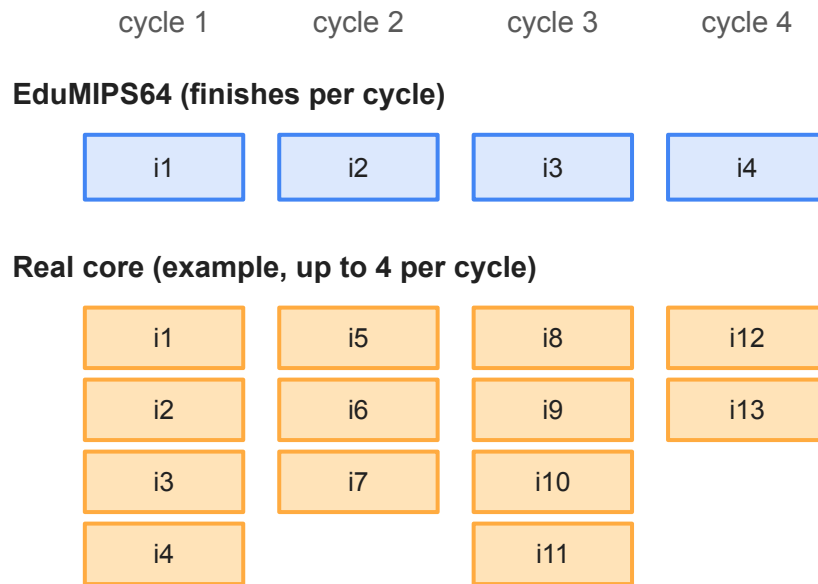
## Part 3

# Learning perf

- 
- Simulator vs real CPU
  - What perf is
  - How it works
  - Reading the output
  - Troubleshooting

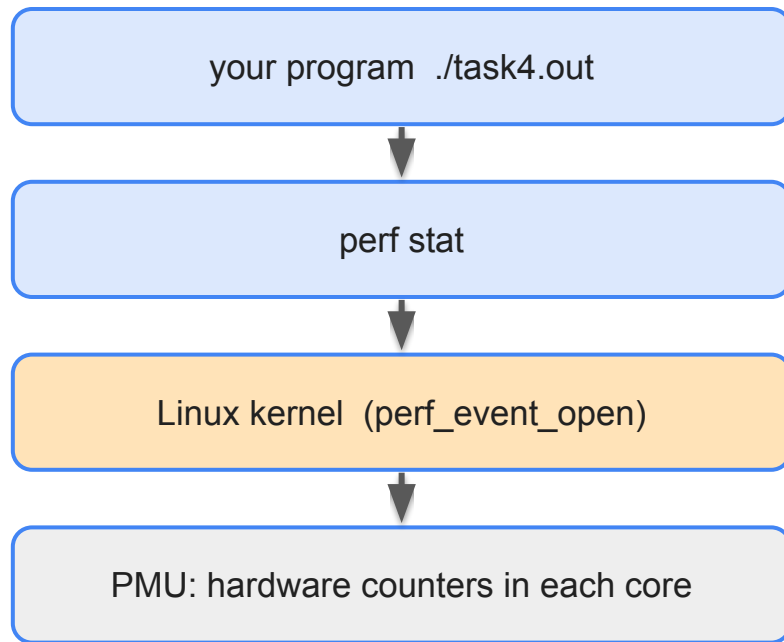
# Welcome to the real world

- EduMIPS64: one instruction per cycle, in order.  $CPI \geq 1$ .
- Real cores are superscalar: they start several instructions per cycle.
- And out of order: an instruction runs as soon as its inputs are ready.
- So  $IPC = \text{instructions} / \text{cycles}$  above 1 is normal on a real CPU.



# Who's counting? perf

- `perf` is a Linux tool that reads the CPU's hardware performance counters.
- The counters live in the PMU (performance monitoring unit) of each core.
- `perf` asks the kernel to set them up and read them for your program.



# perf stat, stat!

| Command                                                                         | What it does                                    |
|---------------------------------------------------------------------------------|-------------------------------------------------|
| <code>perf stat ./prog</code>                                                   | run prog, print a default set of counters       |
| <code>perf stat -e<br/>cycles,instructions,branches,branch-misses ./prog</code> | count only these events                         |
| <code>perf stat -r 5 ./prog</code>                                              | run 5 times, print mean and spread              |
| <code>perf list</code>                                                          | list the events this machine supports           |
| <code>perf record ./prog</code> then <code>perf report</code>                   | sample where time goes (out of scope for today) |

Why perf: counters are exact and cost almost nothing. You see cycles and instructions of the real run.

- **Gold mine for perf usage:** <https://www.brendangregg.com/perf.html>
- Always go back to man page for general usage:  
<https://man7.org/linux/man-pages/man1/perf-stat.1.html>

# Under the hood

---

- Each core has a few hardware counters. Each one counts one event.
- Sometimes (context switch) the kernel saves these counts for your process.
- 2 modes: **Counting mode** (`perf stat`): **totals**. Sampling mode (`perf record`): where it happened.
- What if there are too many events?: perf takes turns (reusing same counter for many events), scales the result by fraction of time it sampled for, and prints a percentage.
- `cycles` are core cycles: they follow the core clock, which **speeds up and slows down**.

Read more: [https://man7.org/linux/man-pages/man2/perf\\_event\\_open.2.html](https://man7.org/linux/man-pages/man2/perf_event_open.2.html)

# Reading perf stat output

```
$ make task4.out
$ perf stat -e task-clock,cycles,instructions \
    ./task4.out 256
```

```
    207.27 msec task-clock
287,175,786      cycles
717,986,882     instructions
```

```
0.210015394 seconds time elapsed
0.204109000 seconds user
0.005973000 seconds sys
```

- task-clock: CPU time the program used
- cycles: core cycles
- instructions: instructions completed
- $IPC = 718M / 287M = 2.5$
- $287M \text{ cycles} / 0.207 \text{ s} = \text{about } 1.4 \text{ GHz}$
- elapsed: wall time; user / sys: time in your code / in the kernel

On the author's laptop.

# Standard events (run `perf list` to see on yours)

| Event                         | Tells you                                                                             |
|-------------------------------|---------------------------------------------------------------------------------------|
| <code>cycles</code>           | core cycles spent                                                                     |
| <code>instructions</code>     | instructions completed; with cycles gives IPC                                         |
| <code>branches</code>         | branch instructions executed                                                          |
| <code>branch-misses</code>    | branches the CPU predicted wrong                                                      |
| <code>cache-references</code> | cache accesses (the exact level depends on the CPU) ( <b>we will see this later</b> ) |
| <code>cache-misses</code>     | of those, how many missed ( <b>we will see this later</b> )                           |
| <code>task-clock</code>       | CPU time used, in ms                                                                  |
| <code>context-switches</code> | times the program was switched out ( <b>we will see this later</b> )                  |
| <code>page-faults</code>      | times the kernel had to map a memory page ( <b>we will see this later</b> )           |

Read more: <https://man7.org/linux/man-pages/man1/perf-list.1.html>

# perf troubleshooting

| Symptom                                                                   | Fix                                                                                                        |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>perf</code> : command not found , or it wants matching kernel tools | <code>sudo apt install linux-tools-common linux-tools-\$(uname -r)</code> (Ubuntu)                         |
| Permission error ( <code>perf_event_paranoid</code> )                     | <code>sudo sysctl -w kernel.perf_event_paranoid=1</code> , or run <code>perf</code> with <code>sudo</code> |
| <code>&lt;not supported&gt;</code> for cycles                             | You are on a VM or WSL without a PMU. Use a real machine :(                                                |
| <code>cycles:u</code> in the output                                       | only user mode is counted. Fine for this lab.                                                              |
| <code>cpu_core/...</code> and <code>cpu_atom/...</code> lines             | You have a hybrid Intel CPU: two kinds of core, counted separately                                         |
| Nothing works                                                             | run on a Software Lab machine (next slide)                                                                 |

# Borrow a real machine

---

- Log in to a Software Lab machine:
  - `ssh <cse-username>@sl<N>-<MM>.cse.iitb.ac.in`
  - Copy the lab folder there (with scp) and run the tasks.
  - Try `perf stat ls` there first. If it prints counters, you are set.
    - How to copy from local to sl machine (use -r if copying whole folder)
- ```
scp [-r] <filename> <cse-username>@sl<N>-<MM>.cse.iitb.ac.in:~/
```

**It's Task 4 time: stat me up**

## Part 4

# Timing code with chrono, rdtsc and perf

---

- `std::chrono`
- The tiny-region problem
- `rdtsc`
- Choosing a clock
- Syscalls (reference only)

# Clock itttt (in C++)

---

- `std::chrono::steady_clock`: never goes backwards. Use it for intervals.
- `std::chrono::system_clock`: the date and time. It can jump.
- `now()` gives a time point. Two time points subtract to a duration.
- `duration<double, std::nano>` turns it into nanoseconds.

```
using clock = std::chrono::steady_clock;
auto t0 = clock::now();
work();
auto t1 = clock::now();
double ns = std::chrono::duration<
    double, std::nano>(t1 - t0).count();
```

Read more: <https://en.cppreference.com/w/cpp/chrono>

# Running out of time

- Reading the clock costs ~20 ns on the author's laptop, while an add takes ~1 ns.
- So **repeat the operation many times then divide**, so timer overhead becomes negligible.
- Store the result in a `volatile` variable (to prevent the compiler from optimizing the work away).
- **Warm up once** before measuring: the first clock read is noticeably slower, subsequent ones are fast.. We will learn why later

| make task5   | result          |
|--------------|-----------------|
| single add   | 20.00 ns        |
| repeated add | 0.43 ns per add |

On the author's laptop, 1,000,000 repetitions.

**It's Task 5 time: no time to lose**

# Low-level stuff: rdtsc

- The time stamp counter (TSC) is a 64-bit counter in the CPU.
- `rdtsc` puts the low 32 bits in `eax`, the high 32 bits in `edx`.
- Note that CPU clock speed is **variable**!
- On current CPUs it ticks at a **fixed rate**, whatever the core clock does (constant or invariant TSC).
- So ticks measure **time but in some other units**.
- **ns = ticks \* 1000 / MHz**. Find your MHz with `make task6`.

| author's laptop            |               |
|----------------------------|---------------|
| TSC rate                   | 3792 MHz      |
| core clock in the perf run | about 1.4 GHz |

Read more: <https://www.felixcloutier.com/x86/rdtsc> · [https://en.wikipedia.org/wiki/Time\\_Stamp\\_Counter](https://en.wikipedia.org/wiki/Time_Stamp_Counter)

# rdtsc cannot be reordered

- Out-of-order execution in a CPU may run `rdtsc` before earlier work has finished.
- Add a `lfence` before it: it fences/waits for all earlier instructions first.
- Use an `rdtscp` for the end timing: it waits for earlier instructions.
- Use `taskset -c <N>` to “pin” your task to Nth core

| Where | Assembly                                     | C, from <x86intrin.h>                                |
|-------|----------------------------------------------|------------------------------------------------------|
| start | <code>lfence</code> then <code>rdtsc</code>  | <code>__mm_lfence(); t0 = __rdtsc();</code>          |
| end   | <code>rdtscp</code> then <code>lfence</code> | <code>t1 = __rdtscp(&amp;aux); __mm_lfence();</code> |

Read more: <https://www.felixcloutier.com/x86/rdtscp> · <https://www.felixcloutier.com/x86/lfence>

# Summary: how to benchmark

---

- Warm up first (do a first run that is not timed) bcz the first run is always slow.
- Repeat, then take the **median/minimum/avg**.
- Measure **long enough** that the clock cost is tiny in comparison.
- Pin to one core: `taskset -c N ./prog`.
- Note CPU frequency can change and heavy programs running in background can affect results.
- To prevent compiler from deleting the variable: mark it **volatile**, and **use it later**.
- Compare like with like: same machine, same flags, same input.
- But which timing mechanism to use?

# Which clock for what

| Tool                       | Measures                                   | Notes                                                                      |
|----------------------------|--------------------------------------------|----------------------------------------------------------------------------|
| <code>perf stat</code>     | whole process: cycles, instructions, time  | no code changes; uses hardware counters                                    |
| <code>std::chrono</code>   | ns between two points in C++               | clock read takes about 20 ns (author's laptop)                             |
| <code>clock_gettime</code> | ns, from C or assembly – this is a syscall | library call takes about 18 ns, raw syscall about 150 ns (author's laptop) |
| <code>rdtsc</code>         | TSC ticks at a fixed rate                  | cheapest read; needs conversion with the MHz factor                        |

Every clock needs a region long enough, or repeated often enough, that the read cost is small.

# What is a syscall? (OPTIONAL)

- A syscall asks the kernel to do something: switch from user mode to kernel mode and back.
- The `syscall` instruction does the switch.
- Number in `rax`. Arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. Result in `rax`.
- It overwrites `rcx` and `r11`.
- Details: `man 2 syscall`.

| syscall       | rax |
|---------------|-----|
| read          | 0   |
| write         | 1   |
| exit          | 60  |
| clock_gettime | 228 |

Read more: <https://man7.org/linux/man-pages/man2/syscall.2.html> · [https://www.youtube.com/watch?v=EU\\_nQh8wg5A](https://www.youtube.com/watch?v=EU_nQh8wg5A)

# clock\_gettime: libc vs raw syscall (OPTIONAL)

---

- libc's `clock_gettime()` normally uses the vDSO: kernel code mapped into your process. It often skips the kernel entry.
- The raw `syscall` always enters the kernel.
- On the author's laptop: about 18 ns vs about 150 ns per call.
- `struct timespec`: `tv_sec` at offset 0, `tv_nsec` at offset 8.
- $ns = (sec1 - sec0) * 1e9 + (nsec1 - nsec0)$
- In a VM the vDSO may fall back to the real syscall.
  
- Watch godbolt talk about benchmarking [here](#)

Read more: <https://man7.org/linux/man-pages/man7/vdso.7.html> · [https://man7.org/linux/man-pages/man2/clock\\_gettime.2.html](https://man7.org/linux/man-pages/man2/clock_gettime.2.html)

**It's Task 6 time: tick, tock, go**

# References (1/2)

---

## EduMIPS64

<https://edumips64.readthedocs.io/en/latest/>  
<https://web.edumips.org>  
<https://github.com/EduMIPS64/edumips64/releases>

## Java

<https://adoptium.net>

## Compilers

<https://godbolt.org>  
<https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

## Pipelining

[https://en.wikipedia.org/wiki/Classic\\_RISC\\_pipeline](https://en.wikipedia.org/wiki/Classic_RISC_pipeline)  
[https://en.wikipedia.org/wiki/Hazard\\_\(computer\\_architecture\)](https://en.wikipedia.org/wiki/Hazard_(computer_architecture))  
[https://en.wikipedia.org/wiki/Operand\\_forwarding](https://en.wikipedia.org/wiki/Operand_forwarding)  
[https://en.wikipedia.org/wiki/Loop\\_unrolling](https://en.wikipedia.org/wiki/Loop_unrolling)  
[https://en.wikipedia.org/wiki/Superscalar\\_processor](https://en.wikipedia.org/wiki/Superscalar_processor)  
[https://en.wikipedia.org/wiki/Out-of-order\\_execution](https://en.wikipedia.org/wiki/Out-of-order_execution)

# References (2/2)

---

## perf

<https://perf.wiki.kernel.org>  
<https://man7.org/linux/man-pages/man1/perf-stat.1.html>  
[https://man7.org/linux/man-pages/man2/perf\\_event\\_open.2.html](https://man7.org/linux/man-pages/man2/perf_event_open.2.html)  
<https://www.brendangregg.com/perf.html>

## Syscalls

<https://man7.org/linux/man-pages/man2/syscall.2.html>  
[https://www.youtube.com/watch?v=EU\\_nQh8wg5A](https://www.youtube.com/watch?v=EU_nQh8wg5A)

## Linux man pages

<https://man7.org/linux/man-pages/>

## Timing

<https://en.cppreference.com/w/cpp/chrono>  
[https://man7.org/linux/man-pages/man2/clock\\_gettime.2.html](https://man7.org/linux/man-pages/man2/clock_gettime.2.html)  
<https://man7.org/linux/man-pages/man7/vdso.7.html>  
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>  
<https://www.felixcloutier.com/x86/rdtsc>